MCA203P — Lab Sheet 05: Solved Java Programs

Object Oriented Programming Using Java | 20 separate programs

Each Java solution is also provided as an individual .java file. The public class name matches its filename. Sample names/data are illustrative.

Q1 — StudentMain.java

```
// Q1: Student Record – Class and Object
class Student {
    String name;
    int rollNo;
    double marks;

    Student() {
        name = "Aarav Sharma";
        rollNo = 101;
        marks = 87.5;
    }

    void displayDetails() {
        System.out.println("Name: " + name);
        System.out.println("Roll No: " + rollNo);
        System.out.println("Marks: " + marks);
    }
}

public class StudentMain {
    public static void main(String[] args) {
        Student student = new Student();
        student.displayDetails();
    }
}
```

Q2 — MedicineMain.java

```
// Q2: Medicine – Parameterized Constructor and this
class Medicine {
    String medicineName;
    String batchNo;
    double price;

    Medicine(String medicineName, String batchNo, double price) {
        this.medicineName = medicineName;
        this.batchNo = batchNo;
        this.price = price;
    }

    void displayMedicine() {
        System.out.println("Medicine: " + medicineName + ", Batch: " + batchNo
                + ", Price: Rs. " + price);
    }
}

public class MedicineMain {
    public static void main(String[] args) {
        Medicine medicine1 = new Medicine("Paracetamol", "PCM101", 25.50);
        Medicine medicine2 = new Medicine("Cetirizine", "CTZ202", 18.00);
        medicine1.displayMedicine();
        medicine2.displayMedicine();
    }
}
```

Q3 — PharmacyMain.java

```
// Q3: Pharmacy – Instance and Class Variables
class Pharmacy {
    String pharmacyName;
    String location;
    static int pharmacyCount = 0; // Shared by every Pharmacy object

    Pharmacy(String pharmacyName, String location) {
        this.pharmacyName = pharmacyName;
        this.location = location;
        pharmacyCount++;
    }

    void displayPharmacy() {
        System.out.println("Pharmacy: " + pharmacyName + ", Location: " + location);
    }

    static void displayPharmacyCount() {
```

```
        System.out.println("Total pharmacies: " + pharmacyCount);
    }
}

public class PharmacyMain {
    public static void main(String[] args) {
        Pharmacy p1 = new Pharmacy("HealthCare Pharmacy", "Roorkee");
        Pharmacy p2 = new Pharmacy("City Medicos", "Haridwar");
        Pharmacy p3 = new Pharmacy("Wellness Pharmacy", "Dehradun");
        p1.displayPharmacy();
        p2.displayPharmacy();
        p3.displayPharmacy();
        Pharmacy.displayPharmacyCount();
    }
}
```

Q4 — PatientMain_Q4.java

```
// Q4: Patient Records — Object Array, Loop and Local Variable
class Patient {
    int patientId;
    String patientName;
    int age;

    Patient(int patientId, String patientName, int age) {
        this.patientId = patientId;
        this.patientName = patientName;
        this.age = age;
    }

    void displayPatient() {
        System.out.println("ID: " + patientId + ", Name: " + patientName + ", Age: " + age);
    }
}

class PatientManager {
    void displayAllPatients(Patient[] patients) {
        for (int i = 0; i < patients.length; i++) { // i is a local loop variable
            patients[i].displayPatient();
        }
    }
}

public class PatientMain {
    public static void main(String[] args) {
        Patient[] patients = {
            new Patient(1, "Aarav", 22),
            new Patient(2, "Diya", 35),
            new Patient(3, "Kabir", 41),
            new Patient(4, "Meera", 28),
            new Patient(5, "Rohan", 53)
        };
        PatientManager manager = new PatientManager();
        manager.displayAllPatients(patients);
    }
}
```

Q5 — EligibilityMain.java

```
// Q5: Medicine Eligibility — Method and Local Variable
class Medicine {
    String medicineName;
    int ageLimit;

    Medicine(String medicineName, int ageLimit) {
        this.medicineName = medicineName;
        this.ageLimit = ageLimit;
    }

    String checkEligibility(int patientAge) {
        String result; // Local variable stores the decision
        if (patientAge >= ageLimit) {
            result = "Eligible";
        } else {
            result = "Not Eligible";
        }
        return result;
    }
}

public class EligibilityMain {
    public static void main(String[] args) {
        Medicine medicine = new Medicine("ExampleMedicine", 18);
        System.out.println("Age 25: " + medicine.checkEligibility(25));
        System.out.println("Age 12: " + medicine.checkEligibility(12));
    }
}
```

Q6 — MedicineProductMain.java

```
// Q6: Encapsulated Medicine Product
class MedicineProduct {
    private String medicineName;
    private String batchNo;
    private double price;

    MedicineProduct(String medicineName, String batchNo, double price) {
        this.medicineName = medicineName;
        this.batchNo = batchNo;
        this.price = price;
    }

    public String getMedicineName() { return medicineName; }
    public String getBatchNo() { return batchNo; }
    public double getPrice() { return price; }

    public void setMedicineName(String medicineName) { this.medicineName = medicineName; }
    public void setBatchNo(String batchNo) { this.batchNo = batchNo; }
    public void setPrice(double price) { this.price = price; }

    public void displayProduct() {
        System.out.println("Medicine: " + medicineName);
        System.out.println("Batch: " + batchNo);
        System.out.println("Price: Rs. " + price);
    }
}

public class MedicineProductMain {
    public static void main(String[] args) {
        MedicineProduct product = new MedicineProduct("Paracetamol", "PCM101", 25.50);
        product.setPrice(29.00);
        System.out.println("Updated price: Rs. " + product.getPrice());
        product.displayProduct();
    }
}
```

Q7 — PatientMain_Q7.java

```
// Q7: Encapsulated Patient – Validation
class Patient {
    private String patientName;
    private int age;
    private double weight;

    Patient(String patientName, int age, double weight) {
        this.patientName = patientName;
        setAge(age);
        setWeight(weight);
    }

    public String getPatientName() { return patientName; }
    public int getAge() { return age; }
    public double getWeight() { return weight; }

    public void setPatientName(String patientName) {
        this.patientName = patientName;
    }

    public void setAge(int age) {
        if (age >= 0) {
            this.age = age;
        } else {
            System.out.println("Invalid age rejected: " + age);
        }
    }

    public void setWeight(double weight) {
        if (weight > 0) {
            this.weight = weight;
        } else {
            System.out.println("Invalid weight rejected: " + weight);
        }
    }

    public void displayPatient() {
        System.out.println("Name: " + patientName + ", Age: " + age + ", Weight: " + weight + " kg");
    }
}

public class PatientMain {
    public static void main(String[] args) {
        Patient patient = new Patient("Ananya", 21, 52.5);
        patient.setAge(22);       // Valid update
        patient.setAge(-4);       // Invalid update
        patient.setWeight(54.0);  // Valid update
        patient.setWeight(0);     // Invalid update
        patient.displayPatient();
```

```
    }
}
```

Q8 — BankAccountMain.java

```
// Q8: Bank Account – Encapsulation, this and Local Variable
class BankAccount {
    private String accountNo;
    private String accountHolderName;
    private double balance;

    BankAccount(String accountNo, String accountHolderName, double balance) {
        this.accountNo = accountNo;
        this.accountHolderName = accountHolderName;
        this.balance = Math.max(0, balance);
    }

    public void deposit(double amount) {
        if (amount > 0) {
            balance += amount;
            System.out.println("Deposit successful: Rs. " + amount);
        } else {
            System.out.println("Deposit failed: amount must be positive.");
        }
    }

    public void withdraw(double amount) {
        boolean success = false; // Local variable tracks the outcome
        if (amount > 0 && amount <= balance) {
            balance -= amount;
            success = true;
        }
        System.out.println(success ? "Withdrawal successful." : "Withdrawal failed.");
    }

    public double getBalance() {
        return balance;
    }

    public void displayAccount() {
        System.out.println("Account No: " + accountNo);
        System.out.println("Account Holder: " + accountHolderName);
        System.out.println("Balance: Rs. " + balance);
    }
}

public class BankAccountMain {
    public static void main(String[] args) {
        BankAccount account = new BankAccount("AC1001", "Aarav Sharma", 5000.00);
        account.deposit(1500.00);
        account.withdraw(1200.00);
        account.displayAccount();
    }
}
```

Q9 — PublicAccessMain.java

```
// Q9: Public Access Specifier
class Hospital {
    // Public members can be accessed from other classes.
    public String hospitalName;
    public String hospitalCode;

    public Hospital() {
        hospitalName = "City Hospital";
        hospitalCode = "CH101";
    }

    public void displayHospital() {
        System.out.println("Hospital: " + hospitalName + ", Code: " + hospitalCode);
    }
}

public class PublicAccessMain {
    public static void main(String[] args) {
        Hospital hospital = new Hospital();
        hospital.hospitalName = "Community Hospital";
        hospital.hospitalCode = "COM202";
        System.out.println(hospital.hospitalName);
        System.out.println(hospital.hospitalCode);
        hospital.displayHospital();
    }
}
```

Q10 — PrivateAccessMain.java

```
// Q10: Private Access Specifier
```

```
class Employee {
    private int employeeId;
    private String employeeName;
    private double salary;

    Employee(int employeeId, String employeeName, double salary) {
        this.employeeId = employeeId;
        this.employeeName = employeeName;
        this.salary = salary;
    }

    public int getEmployeeId() { return employeeId; }
    public String getEmployeeName() { return employeeName; }
    public double getSalary() { return salary; }

    public void displayEmployee() {
        System.out.println("ID: " + employeeId + ", Name: " + employeeName + ", Salary: Rs. " + salary);
    }
}

public class PrivateAccessMain {
    public static void main(String[] args) {
        Employee employee = new Employee(501, "Riya Verma", 42000);
        System.out.println(employee.getEmployeeId());
        System.out.println(employee.getEmployeeName());
        System.out.println(employee.getSalary());
        employee.displayEmployee();

        // System.out.println(employee.salary); // Not allowed: salary is private.
    }
}
```

Q11 — DefaultAccessMain.java

```
// Q11: Default (Package-Private) Access Specifier
class Pharmacy {
    // No keyword means default/package-private access.
    String medicineName;
    double price;

    // Default/package-private constructor.
    Pharmacy() {
        medicineName = "Vitamin C";
        price = 120.00;
    }

    // Default/package-private method.
    void displayMedicine() {
        System.out.println("Medicine: " + medicineName + ", Price: Rs. " + price);
    }
}

public class DefaultAccessMain {
    public static void main(String[] args) {
        Pharmacy pharmacy = new Pharmacy();
        System.out.println(pharmacy.medicineName);
        System.out.println(pharmacy.price);
        pharmacy.displayMedicine();
    }
}
```

Q12 — ProtectedAccessMain.java

```
// Q12: Protected Access Specifier
class HospitalStaff {
    protected int staffId;
    protected String staffName;

    protected HospitalStaff(int staffId, String staffName) {
        this.staffId = staffId;
        this.staffName = staffName;
    }

    protected void displayStaff() {
        System.out.println("Staff ID: " + staffId + ", Name: " + staffName);
    }
}

class Pharmacist extends HospitalStaff {
    private String licenseNo;

    Pharmacist(int staffId, String staffName, String licenseNo) {
        super(staffId, staffName); // Initialize the parent part
        this.licenseNo = licenseNo;
    }

    void displayPharmacist() {
        displayStaff(); // Inherited protected method
```

```
        System.out.println("License No: " + licenseNo);
        System.out.println("Inherited staff ID/name: " + staffId + " / " + staffName);
    }
}

public class ProtectedAccessMain {
    public static void main(String[] args) {
        Pharmacist pharmacist = new Pharmacist(601, "Neha Singh", "PH-7781");
        pharmacist.displayPharmacist();
    }
}
```

Q13 — SingleInheritanceMain.java

```
// Q13: Single Inheritance — Employee -> Pharmacist
class Employee { // Super class
    protected int employeeId;
    protected String employeeName;
    protected double salary;

    Employee(int employeeId, String employeeName, double salary) {
        this.employeeId = employeeId;
        this.employeeName = employeeName;
        this.salary = salary;
    }

    void displayEmployee() {
        System.out.println("Employee ID: " + employeeId);
        System.out.println("Employee Name: " + employeeName);
        System.out.println("Salary: Rs. " + salary);
    }
}

class Pharmacist extends Employee { // Child class
    private String licenseNo;

    Pharmacist(int employeeId, String employeeName, double salary, String licenseNo) {
        super(employeeId, employeeName, salary);
        this.licenseNo = licenseNo;
    }

    void displayPharmacist() {
        displayEmployee();
        System.out.println("License No: " + licenseNo);
    }
}

public class SingleInheritanceMain {
    public static void main(String[] args) {
        Pharmacist pharmacist = new Pharmacist(701, "Karan Mehta", 38000, "PH-9021");
        pharmacist.displayPharmacist();
    }
}
```

Q14 — HierarchicalMain.java

```
// Q14: Hierarchical Inheritance — Person -> Doctor and Nurse
class Person { // Super class
    protected String name;
    protected int age;

    Person(String name, int age) {
        this.name = name;
        this.age = age;
    }

    void displayPerson() {
        System.out.println("Name: " + name + ", Age: " + age);
    }
}

class Doctor extends Person { // Child class 1
    private String specialization;

    Doctor(String name, int age, String specialization) {
        super(name, age);
        this.specialization = specialization;
    }

    void displayDoctor() {
        displayPerson();
        System.out.println("Specialization: " + specialization);
    }
}

class Nurse extends Person { // Child class 2
    private String ward;
```

```
    Nurse(String name, int age, String ward) {
        super(name, age);
        this.ward = ward;
    }

    void displayNurse() {
        displayPerson();
        System.out.println("Ward: " + ward);
    }
}

public class HierarchicalMain {
    public static void main(String[] args) {
        Doctor doctor = new Doctor("Dr. Meera", 38, "Cardiology");
        Nurse nurse = new Nurse("Pooja", 29, "Emergency");
        doctor.displayDoctor();
        nurse.displayNurse();
    }
}
```

Q15 — MultilevelMain.java

```
// Q15: Multilevel Inheritance — Person -> Employee -> Manager
class Person {
    protected String name;
    protected int age;

    Person(String name, int age) {
        this.name = name;
        this.age = age;
    }
}

class Employee extends Person {
    private int employeeId;

    Employee(String name, int age, int employeeId) {
        super(name, age);
        this.employeeId = employeeId;
    }

    int getEmployeeId() {
        return employeeId;
    }
}

class Manager extends Employee {
    private int teamSize;

    Manager(String name, int age, int employeeId, int teamSize) {
        super(name, age, employeeId);
        this.teamSize = teamSize;
    }

    void displayManager() {
        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
        System.out.println("Employee ID: " + getEmployeeId());
        System.out.println("Team Size: " + teamSize);
    }
}

public class MultilevelMain {
    public static void main(String[] args) {
        Manager manager = new Manager("Amit Joshi", 42, 801, 12);
        manager.displayManager();
    }
}
```

Q16 — MedicalStaffMain.java

```
// Q16: Protected Parent + Private Child Data
class MedicalStaff {
    protected int staffId;
    protected String staffName;

    MedicalStaff(int staffId, String staffName) {
        this.staffId = staffId;
        this.staffName = staffName;
    }
}

class Doctor extends MedicalStaff {
    private String specialization;

    Doctor(int staffId, String staffName, String specialization) {
        super(staffId, staffName);
        this.specialization = specialization;
```

```
    }

    void displayDoctor() {
        System.out.println("Doctor ID: " + staffId + ", Name: " + staffName
                + ", Specialization: " + specialization);
    }
}

class Pharmacist extends MedicalStaff {
    private String licenseNo;

    Pharmacist(int staffId, String staffName, String licenseNo) {
        super(staffId, staffName);
        this.licenseNo = licenseNo;
    }

    void displayPharmacist() {
        System.out.println("Pharmacist ID: " + staffId + ", Name: " + staffName
                + ", License: " + licenseNo);
    }
}

public class MedicalStaffMain {
    public static void main(String[] args) {
        Doctor doctor = new Doctor(901, "Dr. Kavita", "Pediatrics");
        Pharmacist pharmacist = new Pharmacist(902, "Rahul Das", "PH-4412");
        doctor.displayDoctor();
        pharmacist.displayPharmacist();
    }
}
```

Q17 — InheritanceConceptMain.java

```
// Q17: Multiple and Hybrid Inheritance — Concept Only
public class InheritanceConceptMain {
    public static void main(String[] args) {
        System.out.println("See source comments for multiple and hybrid inheritance concepts.");
    }

    /*
     * Multiple inheritance means inheriting from more than one parent class.
     * Java classes cannot extend two classes because it can create ambiguity
     * (for example, if both parents define the same method).
     *
     * Invalid Java syntax shown only as a comment:
     * class C extends A, B { }
     *
     * Hybrid inheritance combines two or more inheritance forms, such as
     * hierarchical and multilevel inheritance.
     *
     * Java classes can implement multiple interfaces to support multiple
     * inheritance of type/behavior. This program intentionally does not
     * declare or implement any interface, as required by the question.
     */
}
```

Q18 — ShapeMain.java

```
// Q18: Shape Hierarchy — Parent Reference Array
class Shape {
    protected String name;

    Shape(String name) {
        this.name = name;
    }

    void displayShapeName() {
        System.out.println("Shape: " + name);
    }
}

class Circle extends Shape {
    private double radius;

    Circle(String name, double radius) {
        super(name);
        this.radius = radius;
    }

    double calculateArea() {
        return Math.PI * radius * radius;
    }
}

class Rectangle extends Shape {
    private double length;
    private double width;
```

```
    Rectangle(String name, double length, double width) {
        super(name);
        this.length = length;
        this.width = width;
    }

    double calculateArea() {
        return length * width;
    }
}

class ShapeManager {
    void displayAreas(Shape[] shapes) {
        for (int i = 0; i < shapes.length; i++) { // Local loop variable
            shapes[i].displayShapeName();
            if (shapes[i] instanceof Circle) {
                Circle circle = (Circle) shapes[i];
                System.out.printf("Area: %.2f%n", circle.calculateArea());
            } else if (shapes[i] instanceof Rectangle) {
                Rectangle rectangle = (Rectangle) shapes[i];
                System.out.printf("Area: %.2f%n", rectangle.calculateArea());
            }
        }
    }
}

public class ShapeMain {
    public static void main(String[] args) {
        Shape[] shapes = {
            new Circle("Circle", 5.0),
            new Rectangle("Rectangle", 4.0, 6.0)
        };
        ShapeManager manager = new ShapeManager();
        manager.displayAreas(shapes);
    }
}
```

Q19 — HospitalStaffMain.java

```
// Q19: Hospital Staff Management — Integrated OOP
class Person {
    protected String name;
    protected int age;

    Person(String name, int age) {
        this.name = name;
        this.age = age;
    }
}

class Employee extends Person {
    private int employeeId;
    private double salary;
    static int employeeCount = 0; // Shared count for all Employee objects

    Employee(String name, int age, int employeeId, double salary) {
        super(name, age);
        this.employeeId = employeeId;
        setSalary(salary);
        employeeCount++;
    }

    public int getEmployeeId() { return employeeId; }
    public double getSalary() { return salary; }
    public void setEmployeeId(int employeeId) { this.employeeId = employeeId; }

    public void setSalary(double salary) {
        if (salary >= 0) {
            this.salary = salary;
        }
    }

    void displayBaseRecord() {
        System.out.println("ID: " + employeeId + ", Name: " + name + ", Age: " + age
                + ", Salary: Rs. " + salary);
    }
}

class Doctor extends Employee {
    private String specialization;

    Doctor(String name, int age, int employeeId, double salary, String specialization) {
        super(name, age, employeeId, salary);
        this.specialization = specialization;
    }

    void displayRecord() {
        displayBaseRecord();
        System.out.println("Role: Doctor, Specialization: " + specialization);
    }
```

```
}

class Pharmacist extends Employee {
    private String licenseNo;

    Pharmacist(String name, int age, int employeeId, double salary, String licenseNo) {
        super(name, age, employeeId, salary);
        this.licenseNo = licenseNo;
    }

    void displayRecord() {
        displayBaseRecord();
        System.out.println("Role: Pharmacist, License: " + licenseNo);
    }
}

class HospitalStaffManager {
    void displayStaff(Employee[] staff) {
        for (int i = 0; i < staff.length; i++) {
            if (staff[i] instanceof Doctor) {
                ((Doctor) staff[i]).displayRecord();
            } else if (staff[i] instanceof Pharmacist) {
                ((Pharmacist) staff[i]).displayRecord();
            }
        }
        System.out.println("Total employees: " + Employee.employeeCount);
    }
}

public class HospitalStaffMain {
    public static void main(String[] args) {
        Employee[] staff = {
            new Doctor("Dr. Nitin", 45, 1001, 85000, "Neurology"),
            new Pharmacist("Suman", 31, 1002, 36000, "PH-5582")
        };
        HospitalStaffManager manager = new HospitalStaffManager();
        manager.displayStaff(staff);
    }
}
```

Q20 — PharmacyEmployeeMain.java

```
// Q20: Pharmacy Employee Management System — Final Challenge
class Employee {
    private int employeeId;
    private String employeeName;
    private double salary;
    static int employeeCount = 0; // Class variable shared by all employees
    protected String department = "Pharmacy"; // Inherited by child classes
    String employmentType = "Full-time"; // Default/package-private member

    Employee(int employeeId, String employeeName, double salary) {
        this.employeeId = employeeId;
        this.employeeName = employeeName;
        setSalary(salary);
        employeeCount++;
    }

    public int getEmployeeId() { return employeeId; }
    public String getEmployeeName() { return employeeName; }
    public double getSalary() { return salary; }

    public void setEmployeeId(int employeeId) {
        this.employeeId = employeeId;
    }

    public void setEmployeeName(String employeeName) {
        this.employeeName = employeeName;
    }

    public void setSalary(double salary) {
        if (salary >= 0) {
            this.salary = salary;
        } else {
            System.out.println("Negative salary rejected.");
        }
    }

    void displayEmployee() { // Default-access method
        System.out.println("ID: " + employeeId + ", Name: " + employeeName
                + ", Salary: Rs. " + salary + ", Department: " + department
                + ", Type: " + employmentType);
    }
}

class Pharmacist extends Employee {
    private String licenseNo;

    Pharmacist(int employeeId, String employeeName, double salary, String licenseNo) {
        super(employeeId, employeeName, salary);
```

```
        this.licenseNo = licenseNo;
    }

    void displayRole() {
        displayEmployee();
        System.out.println("Role: Pharmacist, License: " + licenseNo);
    }
}

class StoreManager extends Employee {
    private int storeSection;

    StoreManager(int employeeId, String employeeName, double salary, int storeSection) {
        super(employeeId, employeeName, salary);
        this.storeSection = storeSection;
    }

    void displayRole() {
        displayEmployee();
        System.out.println("Role: Store Manager, Store Section: " + storeSection);
    }
}

class PharmacyEmployeeManager {
    void processEmployees(Employee[] employees) {
        int total = 0; // Local variable used while processing the array
        for (int i = 0; i < employees.length; i++) {
            if (employees[i] instanceof Pharmacist) {
                ((Pharmacist) employees[i]).displayRole();
                total++;
            } else if (employees[i] instanceof StoreManager) {
                ((StoreManager) employees[i]).displayRole();
                total++;
            }
        }
        System.out.println("Processed employees: " + total);
        System.out.println("Total Employee objects created: " + Employee.employeeCount);
    }
}

public class PharmacyEmployeeMain {
    public static void main(String[] args) {
        Employee[] employees = {
            new Pharmacist(2001, "Aditi Sharma", 40000, "PH-7001"),
            new StoreManager(2002, "Vikram Rao", 52000, 3)
        };
        PharmacyEmployeeManager manager = new PharmacyEmployeeManager();
        manager.processEmployees(employees);
    }
}
```